

Software supply chain attacks as a risk in modern software development

Adam Kučík

Faculty of Information Technology

Brno University of Technology

Brno, Czech republic

xkucik00@stud.fit.vut.cz

Abstract—Reusable open-source packages and frameworks are commonly used in modern software development. Although this practice accelerates development and improves efficiency, it also introduces new security risks. Attackers can exploit vulnerabilities or insert malicious code into these dependencies, potentially compromising the software that relies on them. In recent years, software supply chain attacks have become more frequent, highlighting the need for effective methods to detect these threats. This work is meant to provide an overview of software supply chain attacks, explain how attackers exploit open-source projects, present real world consequences of such attacks, and discuss practical tools that help eliminate this threat.

I. INTRODUCTION

Software supply chain (SSC) attacks are becoming a common security threat. The use of open-source projects improves software development efficiency but creates multiple dependencies on external packages, which also contain additional dependencies. According to a Sonatype report, SSC attacks increased by 1,466% between 2014 and 2023 [1]. The most popular packages are targeted as PyPI in python, Node.js in JavaScript, and RubyGems in Ruby. The npm package has 86.55 dependencies on average, while a PyPI Python package defines 7.33 dependencies on average [2].

These attacks target various stages of the software development pipeline. Their goal is to insert malicious code into open-source libraries. This can be achieved by compromising project repositories through social engineering attacks targeting maintainers, or by publishing malicious packages with similar names as the legitimate ones.

In this work, we focus on various ways that attackers use to exploit open-source projects to inject malicious code. We then analyze well known SSC incidents, including SolarWinds hack, Log4j, and the XZ Utils backdoor. Finally, we discuss tools and techniques for mitigating SSC attacks at various stages of the development pipeline, including Software Bill of Materials, dependency checking, and vulnerability scanning.

II. BACKGROUND

An SSC is a connected software development system that relies on components from third parties shared online [3]. open-source projects are typically hosted on platforms such as GitHub and involve maintainers and contributors. Maintainers

are responsible for administering the project, reviewing contributions and approving changes, while contributors are developers who propose changes, add functionality, or otherwise improve the project. Projects often use multiple open-source components as part of the build process [4].

When projects include multiple external components, their dependencies are declared in the project configuration. Dependency managers automatically resolve direct and transitive dependencies to construct a complete dependency tree. These components are then downloaded from package repositories such as PyPI or npm and used during compilation, testing, and development. Such software supply chains introduce multiple trust boundaries. Each dependency creates a potential point of compromise. As the number of dependencies increases, the overall attack surface increases, making supply chain security a significant security challenge [4].

From the attacker's perspective, the goal is to compromise upstream dependencies of the build environment itself to inject malicious code that will propagate to downstream users. Such code can be executed under a specific condition or at a particular time. In the next section, we will provide a detailed overview of the attack options and how to inject malicious code into the user dependency trees to trigger its execution [4].

III. SOFTWARE SUPPLY CHAIN ATTACKS

SSC attacks aim to compromise the cyber infrastructure of a company by targeting software applications or their delivery mechanisms before the company itself receives them. Instead of attacking an organization directly, attackers exploit trusted components within the software development or distribution process. By compromising a dependency, package repository, or build system, malicious code can be introduced into software that is later distributed to many downstream users. The primary objective of an attacker is to inject malicious code into dependency trees. The attackers have two possible strategies; they may either inject an existing package or submit a new package [4].

In the first approach, attackers publish a new package with a name similar to a legitimate one to deceive developers. This technique, for example, is called typosquatting. Alternatively, attackers can develop and promote a package with malware

that appears to provide useful functionality while containing hidden malicious code [4].

The second strategy is to infect an existing package that already has established contributors and maintainers. This can be achieved through social engineering attacks targeting maintainers or by injecting malicious code into the source code or the build process [4].

The following subsections provide an overview of attack techniques, grouped into attacks targeting individuals and attacks targeting software development life cycle. The final subsection discusses the execution of malicious code after it being inserted into dependency trees.

A. Attacks targeting individuals

As in other fields in cybersecurity, individuals play crucial roles throughout the software supply chain, ensuring effective software development, distribution, and consumption. Attackers often exploit human factors to introduce malicious code into software projects. This subsection provides an overview of common methods used to target individuals within the SSC.

Ownership transfer is a technique in which the attacker convinces the project maintainer to become an additional maintainer. This method breaks the implicit trust model of a project by exploiting social vulnerabilities rather than technical flaws. This attack is effective when project's original maintainer is experiencing burnout or is no longer actively engaged, and lacks a formal policy for vetting new contributors. Once the attacker becomes a legitimate maintainer, they can take over the targeted package by removing or replacing existing maintainers. This allows the attacker to introduce malicious code into the package, which can then be propagated to all downstream software projects that rely on it [5].

Account takeover and Phishing. Both approaches aim to obtain the credentials of project maintainers in order to gain unauthorized access to software repositories. In an account takeover attack, attackers exploit weak or reused passwords to access maintainer accounts. In contrast, phishing attacks rely on deceiving maintainers into voluntarily disclosing their credentials in official looking communications. A common approach is to send an anticipated message about security concerns [5].

B. Attacks on the software development lifecycle

This subsection focuses on attacks that target the software development lifecycle by mimicking legitimate packages and exploiting dependency management mechanisms. The following examples describe common techniques used by attackers to introduce malicious code into software projects.

Typosquatting refers to the practice of creating and publishing a malicious package with a name that is a misspelling of a well-known legitimate package, with the intention of misleading users into downloading the malicious one instead of the legitimate one. This was a problem in older versions of npm that allowed anyone to upload packages with any chosen name. Usually, mistyping consists of deleting or inserting

hyphens, or excluding or switching letters. As a result, developers may unintentionally include a malicious dependency without realizing it. Once incorporated into a project, such dependencies can propagate through dependency trees and affect a wide range of downstream applications. Typosquatting is simple to implement and common in practice [5]. Studies have shown that approximately 61% of packages in a selected corpus of software supply chain attacks were associated with this technique [4].

Code injection attack occurs when an attacker injects malicious code into a software project to compromise downstream systems. Attackers often modify only a small portion of the code to make detection more difficult. Once incorporated, malicious code can propagate to end users through normal software distribution channels. A related notable occurrence is ChatGPT Hallucination Hijacking, in which coding assistants can suggest code that, if executed, could introduce malicious behavior into the developer's project [5].

Exploiting unmaintained legacy code. Many projects depend on old projects that are not supported but are still used. The threat is that they can have old well known vulnerabilities, which nobody is going to patch, some of which have assigned Common Vulnerabilities and Exposures (CVE) identifiers. Studies indicate that up to 40 % packages contain at least one known vulnerability, often due to lack of maintenance [5].

Dependency confusion attack exploits an insufficient understanding of developers how software dependencies are managed and resolved. It relies on the fact that dependencies can be sourced from public and private repositories. Libraries or packages are typically obtained from public repositories such as npm and PyPI, while some projects also use privately maintained libraries specific to a company or organization [5].

If a private package is not properly restricted, an attacker can publish a package with the same name to a public repository, often assigning it a higher version number. Due to automatic dependency resolution, the package manager may prioritize the malicious public package over the intended private one. Reports indicate a significant increase in such attacks, with a surge of 7000 % observed in 2021 [5].

Case sensitivity confusion attack occurs when the package manager treats the packages case sensitive in terms of the name of the package. The attacker will register the malicious package with the same name as the legitimate package but with different letter cases. In past npm was allowing names of the packages to be case sensitive, but now it requires all names to be lowercase. Unfortunately, this change simply leaves the popular ones with mixed-cased names of packages leaving vulnerable. For example, the package *buffer* has more than 170 million monthly downloads and continues to coexist with its capitalized variant *Buffer*, because both were published before this policy was enforced [5].

Metadata manipulation attacks are attacks directed against Git to manipulate its metadata stored in the .git directory. Git does not cryptographically sign branch heads or tags by default, which means that there is no way to verify whether the changes were made by an authorized developer.

Although commits and annotated tags can be manually signed, this is an optional feature. Without explicit signing policies, there is no built-in mechanism to verify that metadata changes were made by an authorized actor [5].

Certificate theft is an attack in which adversaries steal developer digital signing certificates that are used to verify that the software is legitimate and trustworthy. By obtaining these certificates, attackers can sign malicious updates and distribute them as if they were authentic [5].

This attack is often caused by improper storage or protection of signing keys. Stolen certificates can also be sold on the black market, allowing other attackers to distribute malware that can affect a wide range of end users [5].

Package version downgrade attack. The attacker's goal is to lower the version of a package to an older one with some known vulnerability. Developers often define version constraints in dependency managers to avoid insecure versions. Manipulation of these constraints may still result in the installation of a vulnerable version without triggering errors. This attack is more likely to occur in projects with a large number of dependencies, particularly when the dependency tree becomes complex. A common countermeasure against this attack is version pinning, which enforces the use of a specific, trusted version of a package [5].

Compromised tools in software development refer to situations in which IDEs, compilers, or build tools were altered by attackers. This alteration can lead to unauthorized activities such as data theft, backdoor installation, or further spread of malware within a network. This threat often arises when developers download tools from unverified sources or fail to verify the integrity of software using checksums or digital signatures [5].

C. Execution of malicious code

After successfully injecting malicious code into a dependency, attackers rely on various mechanisms to trigger its execution. In many cases, malicious code is executed during the installation phase, for example, through package manager scripts, or during the build and deployment process. In other scenarios, the code may remain hidden and activate only under specific conditions, such as system configurations or user actions [4].

To avoid detection, attackers often employ techniques such as code obfuscation [6] or delayed execution. Once executed, malicious code can perform a range of harmful actions, including data exfiltration, establishing backdoors, or compromising additional systems within the network [4].

The following section presents real world examples of successful SSC attacks that resulted in security incidents.

IV. REAL WORLD EXAMPLES

This section presents selected real world example of SSC attacks, including SolarWinds, XZ Utils, and Log4j, highlighting their impact and security implications.

A. SolarWinds

The SolarWinds incident is one of the most significant early examples of a SSC attack. It targeted a US software company with more than 18,000 customers, many US government agencies, fortune 500 companies, and organizations worldwide. During 2019-2020, the company was the target of a highly sophisticated attack by the Russian hacker group CozyBear. The attack involved inserting malicious code into updates to the Orion platform, developed by SolarWinds. Because these updates were digitally signed and distributed through official channels, they appeared legitimate to customers. The attack was discovered by the cybersecurity company FireEye after they found that hackers had gained access to their network through a vulnerability in the Orion software [7].

The attack worked by creating a backdoor in the Orion software updates. This backdoor, named SUNBURST, was introduced by the SUNSPOT malware through the *taskhostsvc.exe* process, which was used to inject a malicious Sunburst DLL file signed with the SolarWinds certificate [8]. Malware enabled attackers to selectively communicate with the compromised system, while remaining stealthy, as it activated only on chosen targets and avoided widespread detection [7].

It is not possible to accurately estimate the extent of the damage caused by this attack. Hackers were able to steal sensitive data, including telemetry, trade, and government secrets, for a significant amount of time, approximately 9 months [7]. Some reports estimate SolarWinds total costs at around 90 million [9].

This incident highlights why SSC security is critical. Even organizations with strong internal defenses can be compromised. The aftermath of this attack was a wake up call to improve cybersecurity practices and increase resilience in the face of sophisticated cyber threats. The SolarWinds attack demonstrated that attackers can exploit trust relationships, effectively turning legitimate software into a powerful attack vector [7].

B. XZ Utils

The XZ Utils incident represents an example of a SSC attack that used social engineering from the attackers perspective to become a maintainer. The attack was done on XZ Utils, which is a widely used data compression library used in many Linux distributions and critical components. In 2024, a malicious backdoor was discovered in the development versions of the library [10].

The attack was performed by a long term contributor operating under the pseudonym Jia Tan, who became involved in the project's maintenance in 2021, gradually establishing a position of trust within the community before executing the attack. The backdoor was designed to target the OpenSSH authentication mechanism, allowing unauthorized remote access under specific conditions [10].

The vulnerability was discovered, by luck, beginning with observation of one PostgreSQL developer, who noticed unusually high CPU usage by ssh processes during SSH logins.

Fortunately, this incident was identified before widespread deployment in stable releases, limiting its overall impact [10].

This incident demonstrated how attackers can exploit trust in open-source communities and maintainers to introduce malicious functionality into widely used software components. This case highlights the danger of social engineering attacks within SSC.

C. Log4j

The Log4j incident is not a traditional supply chain attack, it is more closely related to vulnerability, but many articles include it in the SSC attacks because it demonstrates the catastrophic systemic risk of dependency chains. This incident was scored 10.0 by NIST highlighting the maximum severity rating [11].

Log4j is an open-source library for logging in Java. It is developed by Apache Software Foundation. The vulnerability was introduced in 2013 when a member of the open-source community requested to add a Java Naming and Directory Interface lookup feature to ease data storage. The same person developed the feature in a path that accidentally contained vulnerable code. By injecting specially constructed log messages, attackers could trigger remote requests to external servers, leading to unauthorized code execution without requiring authentication [8], [12].

The vulnerability was discovered by an Alibaba security engineer, who contacted Apache Software Foundation. While Apache Software Foundation was creating the fix, they notified that the vulnerability was shared online on blog, twitter, and GitHub. After this, the Apache Software Foundation released CVE-2021-44228, which was quickly abused by attackers. After this CVE there were also additional vulnerable discoveries, which attackers tried to abuse for over year until 2022 [8], [13].

This incident highlights how quickly vulnerabilities in widely used components can escalate into global security threats. It demonstrates the challenges of coordinated disclosure and patching.

In this section, we describe some significant attacks in SSC. As described in this article, this type of attack is becoming more frequent. During the writing of this article there were two bigger new SSC attacks. The first was aimed at the Python based package used in AI based systems LiteLLM [14]. The second targeted package was the JavaScript library named Axios [15]. Both of these attacks were performed using social engineering methods.

These recent incidents demonstrate that SSC attacks are not isolated events, but an ongoing and rapidly evolving threat. This motivates the need for effective mitigation techniques and defensive mechanisms to reduce the risk of such attacks. In the following section, we will present strategies that help us defend ourselves against SSC attacks.

V. MITIGATION STRATEGIES

This section provides the theoretical background behind the mitigation strategies used to deal with SSC attacks, focusing on dependencies, build processes, and trust relationships.

A. Software Bill of Materials

A Software Bill of Materials (SBOM) is a structured formal list of all components, libraries, and dependencies included in a software system. Provides detailed information about each component, such as version numbers, licenses, and relationships between dependencies. SBOMs are typically designed to be machine-readable and based on standardized formats such as SPDX, CycloneDX, and SWID [16], [17].

The idea and concepts behind SBOM were not new and have seen advances since 2018 [18], gaining increased attention following major software supply chain incidents such as SolarWinds. This exposed how risky hidden dependencies can be. A vulnerable library can open the door to attackers. SBOM helps with this, because it improves visibility into the SSC. By maintaining an accurate inventory of components, the organization can easily identify vulnerable or compromised dependencies and respond to newly discovered vulnerabilities [16], [17].

The main advantage is that companies can easily see the layers of their software that are layered and rapidly changing. Provides stronger security and faster response. For example, SBOMs teams were able to quickly mitigate the Log4j version [16].

Despite these benefits, SBOM adoption presents several challenges. These include scalability issues, increased complexity in CI/CD pipelines, and the need for expertise in managing SBOM tools and formats. Nevertheless, SBOMs are increasingly becoming a standard practice for improving transparency, security, and trust in modern software development [16].

B. Dependency Management

Dependency management is crucial to effectively manage dependencies to maintain the security and integrity of the SSC. Improper handling of dependencies can introduce vulnerabilities or malicious code into a system [5].

An important practice is the removal of obsolete or unused dependencies, as they increase the surface of the attack and can introduce risks through the legacy library [5].

Another key technique is version pinning, which prevents automatic upgrades or downgrades of dependencies, which may contain vulnerability or malicious modifications. Version pinning also helps mitigate dependency confusion attacks, where attackers exploit the interaction between private and public package repositories by publishing malicious packages with the same name as internal dependencies [5].

C. Secure Development

Secure development practices are essential in all areas of software development. Widely used systems, such as Linux, form the backbone of open-source ecosystems, making them attractive targets for attackers due to their widespread adoption [19].

Generally, there exist frameworks such as the Secure Software Development Framework, which emphasize the integra-

tion of security throughout the software lifecycle rather than treating it as a post-development step [20].

A common and low cost technique is manual code reviews. These reviews reduce the SSC risks and can also act as both detective controls and deterrents. Reviews should be performed by peers who have both experience in the technology being used and in secure coding processes. Reviewers should look for both unintentional security flaws and intentional code that could serve malicious purposes. The results of the reviews should be documented for future potential reviews [21].

D. Vulnerability Detection and Monitoring

A common approach to vulnerability detection in software supply chains is Software Composition Analysis (SCA), which focuses on identifying and analyzing third-party components used within an application. Software Composition Analysis tools compare software dependencies with vulnerability databases that contain CVE entries [22], [23].

In modern development environments, Software Composition Analysis tools are integrated into CI/CD pipelines. This enables automated scan of dependencies during the build process, ensuring that vulnerable components are identified before deployment [22]. Tools such as OWASP Dependency-Check are used to automate this process and provide warnings about known vulnerabilities [23].

Continuous monitoring complements this process by tracking newly disclosed vulnerabilities over time, as previously secure applications may become vulnerable as new security issues are published [22].

In addition to Software Composition Analysis, other approaches to vulnerability detection aim to identify injected or malicious behavior during build-time or run-time. These methods focus on verifying whether a software component behaves as expected or exhibits characteristics of malicious code [24].

Such approaches can also be categorized into statistical and dynamic techniques. Statistical methods evaluate software versions and artifacts by comparing them with reference data. They often rely on machine learning techniques, anomaly detection, or clustering applied to metadata and source code features to identify potentially malicious packages [24].

Dynamic techniques, in contrast, execute software components in isolated environments and observe their runtime behavior. The observed behavior is then compared with the expected execution patterns to detect anomalies or hidden malicious functionality [6].

A limitation of statistical methods is their tendency to produce a higher number of false positives, which requires additional manual verification of flagged results. Dynamic approaches, while generally more precise in identifying malicious behavior, may introduce higher computational overhead due to runtime analysis [6], [24].

E. Large Language Models and SSC

Large language models (LLMs) are increasingly integrated into software development lifecycles, where they are used for

tasks such as code generation, completion, and evaluation. Although they are not traditional software dependencies, they influence the software supply chain by affecting how code is produced and introduced into systems. As they are useful in software development, they can be used as a potential attack vector [25].

Various LLMs have revolutionized coding tasks, including code completion and code evaluation. Although they are capable of code generation, they also suffer from insecure code generation. Although LLMs models code can introduce insecure implementations, some research suggests that its vulnerability rate may be lower than that of code sourced from common developer platforms such as Stack Overflow [25].

From a security perspective, LLMs introduce new risks into the software supply chain. These include the generation of insecure code, the reliance on potentially biased or low-quality training data, and the exploitation of a new type of prompt injection attack. In such cases, adversaries can manipulate model inputs or training data to influence the output and introduce security flaws. At the same time, LLMs can also be used for defensive purposes, such as vulnerability detection and code analysis, highlighting their dual role in software security [25].

In general, the integration of LLMs into software development introduces new trust assumptions and security challenges that must be carefully considered in software supply chain security.

These strategies are often implemented using specialized tools that automate dependency analysis, vulnerability detection, and SBOM generation. The following section presents practical examples of such tools and their usage.

VI. PRACTICAL TOOLS

In this section, we discuss practical tools used to defend against and mitigate the impact of software supply chain attacks. The focus is on dependency analysis tools (Snyk and OWASP Dependency-Check), SBOM generation (Syft), and vulnerability scanning (Grype).

A. OWASP Dependency-Check

OWASP Dependency-Check is an open-source tool designed to identify the use of known vulnerable components in software projects. It addresses risks related to vulnerable and outdated dependencies, such as the OWASP TOP 10 2021 entry. The tool scans dependencies and collects information, named as evidence about each component. Each evidence collects information about the vendor, the product, and the version. Then, this evidence is compared with the National Vulnerability Database (NVD), which contains standardized CVE entries [26].

Based on the evidence collected, the tool attempts to match dependencies with known vulnerable components with the list of CVE entries from NVD. Based on the severity of the vulnerability, it rates the evidence in different confidence levels – low, medium, high, and highest. Due to the nature of dependency matching, the tool may produce both false

positives and false negatives. As a result, manual verification is required [26].

B. Snyk

Snyk is a platform designed to identify, prioritize, scan, and fix code security vulnerabilities. It is used for multiple security use cases, but we will focus just on dependency scanning, for open-source packages, as it is comparable to tools used against SSC attacks [27].

Snyk Open Source is providing SCA based solution that analyzes project dependencies and compares them against vulnerable databases. Snyk is used via its command line interface (CLI) or integrated into CI/CD pipelines [27].

In practice, Snyk serves a similar purpose as OWASP Dependency-Check, as both tools focus on identifying vulnerable dependencies in software projects. However, Snyk provides a more integrated workflow, including official support for IDE plugins, such as Eclipse, which allows developers to view vulnerability information directly within the development environment [27], [28].

C. Syft

Syft is an open-source tool that is used to generate SBOMs from container images and file systems. It provides visibility into software dependencies by identifying installed packages and libraries [29].

The tool supports SSC security by improving the transparency of software components, which is used to detect hidden or vulnerable dependencies. Syft can generate SBOMs in standardized formats such as SPDX and CycloneDX [29], [30].

Syft is commonly used through a CLI, where it analyzes a container image or directory and outputs a structured list of all detected components [30]. This SBOM can then be used for further vulnerability analysis in tools such as Gripe [29].

D. Gripe

Gripe is an open-source CLI tool that is used to analyze container images, filesystems, and SBOMs to detect known vulnerabilities [31].

It works by comparing identified software components with vulnerability databases that contain CVE entries. It uses this information to determine whether a given package is affected by a known security issue [31].

The Gripe output provides information on detected vulnerabilities, including the package name, its version, the fixed version (if available), and a severity-based risk assessment [31].

VII. CONCLUSION

This article examined software supply chain attacks, including the techniques used by attackers and their increasing impact on modern software systems. It also discussed recent real world incidents that highlight the severity of this threat, followed by an overview of mitigation strategies and practical tools that can be used to reduce associated risks.

Personally, I find this to be a pretty creative way for the attacker to attack the software. The main advantage from the attackers perspective is that they do not need to distribute the vulnerability between the victim, but that is done by the popularity or trust of an open-source library or some company that distributes software to many global companies.

A key challenge for developers is defending their projects when vulnerabilities appear in the fundamental components on which their systems are based. In such cases, developers have limited ability to directly address the issue before it is resolved by the maintainers. As a result, it forces developers to implement a new layer of security measures and thus expand their knowledge of how to use these tools, which is not popular.

In conclusion, software supply chain attacks are likely to remain a persistent and significant threat in modern software development. As their prevalence increases, widely used open-source projects should adopt and maintain stronger security practices. At the same time, individual organizations should develop their own security measures to reduce the potential impact of such attacks on their systems and customers.

REFERENCES

- [1] Sonatype. (2023) 10th annual state of the software supply chain. [Online]. Available: <https://www.sonatype.com/state-of-the-software-supply-chain/2024/10-year-look>
- [2] M. Ohm, A. Sykosch, and M. Meier, "Towards detection of software supply chain attacks by forensic artifacts," in *Proceedings of the 15th international conference on availability, reliability and security*, 2020, pp. 1–6.
- [3] Sonatype. (2026) What is a software supply chain? [Online]. Available: <https://www.sonatype.com/resources/articles/what-is-software-supply-chain>
- [4] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," 2020. [Online]. Available: <https://arxiv.org/abs/2005.09535>
- [5] B. Gokkaya, L. Aniello, and B. Halak, "Software supply chain: A taxonomy of attacks, mitigations and risk assessment strategies," *Journal of Information Security and Applications*, vol. 97, p. 104324, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2214212625003606>
- [6] X. Zheng, C. Wei, S. Wang, Y. Zhao, P. Gao, Y. Zhang, K. Wang, and H. Wang, "Towards robust detection of open source software supply chain poisoning attacks in industry environments," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. ACM, Oct. 2024, p. 1990–2001. [Online]. Available: <http://dx.doi.org/10.1145/3691620.3695262>
- [7] N. Meegammana and V. N. P., "Solarwinds attack 2020: A cyber security and legal perspective," 11 2025.
- [8] S. Hamer, J. Bowen, M. N. Haque, R. Hines, C. Madden, and L. Williams, "Closing the chain: How to reduce your risk of being solarwinds, log4j, or xz utils," 2025. [Online]. Available: <https://arxiv.org/abs/2503.12192>
- [9] R. Alkhadra, J. Abuzaid, M. AlShammari, and N. Mohammad, "Solar winds hack: In-depth analysis and countermeasures," in *2021 12th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2021, pp. 1–7.
- [10] P. Przymus and T. Durieux, "Wolves in the repository: A software engineering analysis of the xz utils supply chain attack," 2025. [Online]. Available: <https://arxiv.org/abs/2504.17473>
- [11] TechNova. (2024) You locked the front door, but invited the thief through the update: Supply chain attacks explained. [Online]. Available: https://dev.to/technova_768/you-locked-the-front-door-but-invited-the-thief-through-the-update-supply-chain-attacks-explained-3lc4
- [12] S. Anasuri, "Secure software supply chains in open-source ecosystems," *International Journal of Emerging Trends in Computer Science and Information Technology*, vol. 4, no. 1, pp. 62–74, 2023.

- [13] J. Doll, C. McCarthy, H. McDougall, and S. Bhunia, "Unraveling log4shell: Analyzing the impact and response to the log4j vulnerabil," 2025. [Online]. Available: <https://arxiv.org/abs/2501.17760>
- [14] A. Martin. (2026) Supply chain attack hits widely-used ai package, risks impacting thousands of companies. [Online]. Available: <https://therecord.media/supply-chain-attack-hits-widely-used-ai-package>
- [15] Axios. (2026) North korean hackers implicated in major supply chain attack. [Online]. Available: <https://www.axios.com/2026/03/31/north-korean-hackers-implicated-in-major-supply-chain-attack>
- [16] GitHub. (2025) What is a software bill of materials (sbom)? [Online]. Available: <https://github.com/resources/articles/what-is-an-sbom-software-bill-of-materials>
- [17] M. G. Tasmiha Khan. (2026) What is a software bill of materials (sbom)? [Online]. Available: <https://www.ibm.com/think/topics/sbom>
- [18] Cisa. (2026) Software bill of materials (sbom). [Online]. Available: <https://www.cisa.gov/sbom>
- [19] R. Timalisina. (2025) 2025 open-source security: What it is, best practices & more. [Online]. Available: <https://tuxcare.com/blog/open-source-security/>
- [20] NIST. (2025) Secure software development framework. [Online]. Available: <https://csrc.nist.gov/projects/ssdf>
- [21] OWASP. (2026) Software supply chain security. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Software_Supply_Chain_Security_Cheat_Sheet.html
- [22] Sonatype. (2026) Software composition analysis (sca). [Online]. Available: <https://www.sonatype.com/resources/articles/what-is-software-composition-analysis>
- [23] OWASP. (2026) Owasp dependency-check. [Online]. Available: <https://owasp.org/www-project-dependency-check/>
- [24] M. Ohm and C. Stuke, "Sok: Practical detection of software supply chain attacks," in *Proceedings of the 18th International Conference on Availability, Reliability and Security*, 2023, pp. 1–11.
- [25] L. Williams, G. Benedetti, S. Hamer, R. Paramitha, I. Rahman, M. Tamanna, G. Tystahl, N. Zahan, P. Morrison, Y. Acar *et al.*, "Research directions in software supply chain security," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 1–38, 2025.
- [26] OWASP, *OWASP Dependency-Check documentation*, 2026, includes General section, How it Works section and Project Presentation (pdf). [Online]. Available: <https://dependency-check.github.io/DependencyCheck/>
- [27] Snyk, *Snyk CLI for Open Source*, 2026, includes What's Snyk? section for general information about Snyk in documentation. [Online]. Available: <https://docs.snyk.io/developer-tools/snyk-cli/scan-and-maintain-projects-using-the-cli/snyk-cli-for-open-source>
- [28] —. (2026) Third-party dependency scanning (sca, snyk open source). [Online]. Available: <https://docs.snyk.io/developer-tools/snyk-ide-plugins-and-extensions/eclipse-plugin/third-party-dependency-scanning-sca-snyk-open-source>
- [29] Syft. (2026) Syft. [Online]. Available: <https://github.com/anchore/syft>
- [30] —. (2026) Sbom generation. [Online]. Available: <https://oss.anchore.com/docs/guides/sbom/>
- [31] Grype. (2026) Vulnerability scanning. [Online]. Available: <https://oss.anchore.com/docs/guides/vulnerability/>